

BANA 4373: Advanced Business Analytics for Economics and Business

Lecture 2: Data Cleaning, Reproducibility, and Your First API Call (Census)

Dr. Fidel González

Sam Houston State University

Spring 2026

Today's Goals

By the end of today, you should be able to:

- Explain why **data cleaning** is central to real-world analytics
- Recognize common problems in economic datasets (missingness, units, IDs)
- Describe what makes an analysis **reproducible**
- Make your **first API request** to the U.S. Census and load the result into a dataframe

The Reality of Data Work

- Most mistakes in applied work are **not** from fancy models
- Most mistakes come from:
 - dirty data
 - undocumented assumptions
 - irreproducible workflows
- **Models amplify errors.** They don't fix them.

Data Cleaning Is an Analytical Decision

- Cleaning is not just “formatting”
- Cleaning choices embed assumptions:
 - Which observations count?
 - How do we treat missing values?
 - Which definitions apply (metadata)?
- In economics and policy, these choices can change conclusions

Common Problems in Economic Data

- Missing values (not random!)
- Inconsistent units (percent vs. level; nominal vs. real)
- Changing definitions over time (metadata!)
- Duplicate observations
- Mismatched identifiers:
 - FIPS (counties), NAICS (industries), firm IDs

Missing Data: Why It Happens

- Survey nonresponse
- Confidentiality suppression
- Reporting thresholds
- Data collection errors

Key idea: before choosing a method, ask why the data is missing.

Handling Missing Data (Conceptually)

- Drop missing observations (complete-case)
- Simple imputation (mean/median/last observation)
- Missingness indicators (flag missing as information)

Each approach changes the dataset — and may change your conclusion.

Reproducibility: What It Means

A result is reproducible if:

Someone can start from raw data and obtain your results using your code.

- No manual spreadsheet edits
- Clear folder structure
- Scripted cleaning and analysis
- Documented assumptions

A Basic Reproducible Pipeline

- **Raw data** (never modified)
- **Cleaning scripts** (create analysis-ready data)
- **Analysis scripts** (models, figures, tables)
- **Outputs** (final charts, tables, memo/dashboard)

Goal: your future self should be able to reproduce your work in 5 minutes.

Transition: Your First API Call

- Many economic datasets are available via **APIs**
- API benefits:
 - up-to-date data
 - fewer manual errors
 - fully reproducible data pulls
- Today we use the **U.S. Census API (ACS)**

Census Developers Page (API Docs)

API Demo (No Code First)

Step 1: Open this URL in your browser (copy/paste):

```
https://api.census.gov/data/2022/acs/acs5  
?get=NAME,B19013_001E,B01003_001E  
&for=state:48
```

- B19013_001E = Median household income (estimate)
- B01003_001E = Total population (estimate)
- state:48 = Texas (FIPS 48)

You will see JSON — a table in disguise.

What Is JSON? (API Output → DataFrame)

JSON (what APIs return)

- Text-based data format
- Organized as lists and key–value pairs
- Easy for computers to send and receive

```
[  
  ["NAME", "B19013_001E", "state"],  
  ["Texas", "67404", "48"]  
]
```

DataFrame (what analysts want)

- Tabular structure (rows + columns)
- Numeric types understood correctly
- Ready for analysis and visualization

NAME	Income	State
Texas	67,404	48

APIs send JSON → we convert it into a DataFrame for analysis.

What You Should Notice in the Browser Output

- The first row contains **column names**
- Each following row is a **record**
- Values come as text and must often be converted to numbers

This is exactly what Python will read.

Now the Same Thing in Python (Live)

Goal: Make a request, then load into a dataframe.

```
import requests
import pandas as pd

url = ("https://api.census.gov/data/2022/acs/acs5"
       "?get=NAME,B19013_001E,B01003_001E"
       "&for=state:48")

r = requests.get(url, timeout=30)
r.raise_for_status()
data = r.json() # list of rows
data[:2]        # header + first data row
```

Convert the API Result to a DataFrame

```
header = data[0]
rows = data[1:]

df = pd.DataFrame(rows, columns=header)

df["B19013_001E"] = pd.to_numeric(df["B19013_001E"], errors="coerce")
df["B01003_001E"] = pd.to_numeric(df["B01003_001E"], errors="coerce")

df
```

Notice: we had to explicitly convert strings to numbers.

A Slightly Richer Example: Texas Counties

```
https://api.census.gov/data/2022/acs/acs5  
?get=NAME,B19013_001E,B01003_001E  
&for=county:*  
&in=state:48
```

- `for=county:*` asks for **all counties**
- `in=state:48` restricts to Texas
- Same variables: income + population

Python for the County Example

```
url = ("https://api.census.gov/data/2022/acs/acs5"  
      "?get=NAME,B19013_001E,B01003_001E"  
      "&for=county:*&in=state:48")  
  
data = requests.get(url, timeout=30).json()  
df = pd.DataFrame(data[1:], columns=data[0])  
  
df["B19013_001E"] = pd.to_numeric(df["B19013_001E"], errors="coerce"  
    )  
df["B01003_001E"] = pd.to_numeric(df["B01003_001E"], errors="coerce"  
    )  
  
df.sort_values("B19013_001E", ascending=False).head(10)
```

Connecting Back to Reproducibility

- This API call is **reproducible**:
 - same URL
 - same variables
 - same geography
- No manual download steps
- Easy to rerun if data updates or you change states/years

Looking Ahead

- Next class: cleaning real economic datasets (joins, missingness, IDs)
- Homework will emphasize:
 - clean ETL steps
 - documentation
 - reproducibility

Key Takeaways

- Cleaning is an analytical decision
- Missing data is rarely random
- Reproducibility is credibility
- APIs make your workflow faster and more trustworthy